

# Implementing Interrupts, Timers, and Memory- Mapped I/O in CREATOR

Trabajo de Fin de Máster

**uc3m**

Universidad  
**Carlos III**  
de Madrid

---

Autor: Luis Daniel Casais Mezquida

Tutor: Alejandro Calderón Mateos

02 de marzo de 2026

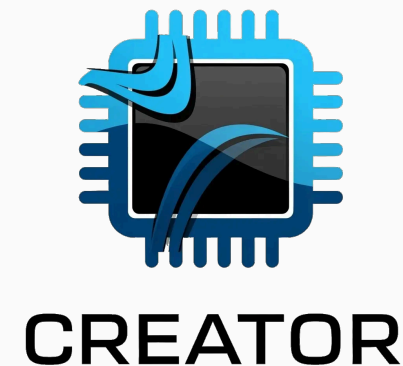
Máster en Ingeniería Informática – Universidad Carlos III de Madrid

# 1. Introducción

---

## 1.1 Motivación

- Aprender lenguaje ensamblador es importante
  - *Software* se construye sobre *hardware*
  - Ciberseguridad, eficiencia, etc.
- Interrupciones son la base del *hardware*
  - Temporizadores, E/S
  - S.O., GPUs, NPUs
- Falta de simuladores con soporte
- *Entorno de desarrollo integrado para formación e investigación en procesadores RISC (PDC2023-145832-I00)*
  - Soporte interrupciones, temporizadores, dispositivos E/S en **CREATOR**
  - Modernizar la aplicación



## **2. Estado de la cuestión**

---

# 2.1 CREATOR

## *didactic and generic assembly programming simulator*

**CREATOR** View Tools Help [Reset] [Step] [Run] [Stop] [Sentinel] [Settings] [Notifications] [Help]

| Address                        | User Instruction           | Loaded Instructions    |
|--------------------------------|----------------------------|------------------------|
| 0x0 <b>main</b>                | la a0, attacker_payload    | auipc a0, 512          |
| 0x4                            |                            | addi a0, a0, 0         |
| 0x8                            | jal ra, process_user_input | jal ra, 12 <b>next</b> |
| 0xC                            | li a7, 10                  | addi a7, x0, 10        |
| 0x10                           | ecall                      | ecall                  |
| 0x14 <b>process_user_input</b> | addi sp, sp, -32           | addi sp, sp, -32       |
| 0x18                           | sw ra, 28(sp)              | sw ra, 28(sp)          |
| 0x1C                           | mv t0, sp                  | addi t0, sp, 0         |
| 0x20                           | mv t1, a0                  | addi t1, a0, 0         |
| 0x24                           | li t2, 8                   | addi t2, x0, 8         |
| 0x28 <b>copy_loop</b>          | beqz t2, copy_done         | beq t2, x0, 28         |
| 0x2C                           | lw t3, 0(t1)               | lw t3, 0(t1)           |
| 0x30                           | sw t3, 0(t0)               | sw t3, 0(t0)           |
| 0x34                           | addi t1, t1, 4             | addi t1, t1, 4         |
| 0x38                           | addi t0, t0, 4             | addi t0, t0, 4         |
| 0x3C                           | addi t2, t2, -1            | addi t2, t2, -1        |
| 0x40                           | j copy_loop                | beq zero, zero, -24    |
| 0x44 <b>copy_done</b>          | lw ra, 28(sp)              | lw ra, 28(sp)          |
| 0x48                           | addi sp, sp, 32            | addi sp, sp, 32        |
| 0x4C                           | jr ra                      | jalr x0, 0(ra)         |
| 0x50 <b>malicious</b>          | li t5, 0xC0FFEE01          | lui t5, 790527         |
| 0x54                           |                            | addi t5, t5, -511      |

**Registers** [Memory] [Terminal] [Statistics]

**Control registers** [Hex]

| pc       | mepc     | mcause   | mtvec    | mstatus  |
|----------|----------|----------|----------|----------|
| 00000008 | 00000000 | 00000000 | 00000000 | 00000008 |

**Integer registers** [Hex]

| x0   zero | x1   ra   | x2   sp   | x3   gp      | x4   tp  |
|-----------|-----------|-----------|--------------|----------|
| 00000000  | FFFFFFF0  | 0FFFFFFC  | 00000000     | 00000000 |
| x5   t0   | x6   t1   | x7   t2   | x8   fp   s0 | x9   s1  |
| 00000000  | 00000000  | 00000000  | 00000000     | 00000000 |
| x10   a0  | x11   a1  | x12   a2  | x13   a3     | x14   a4 |
| 00200000  | 00000000  | 00000000  | 00000000     | 00000000 |
| x15   a5  | x16   a6  | x17   a7  | x18   s2     | x19   s3 |
| 00000000  | 00000000  | 00000000  | 00000000     | 00000000 |
| x20   s4  | x21   s5  | x22   s6  | x23   s7     | x24   s8 |
| 00000000  | 00000000  | 00000000  | 00000000     | 00000000 |
| x25   s9  | x26   s10 | x27   s11 | x28   t3     | x29   t4 |
| 00000000  | 00000000  | 00000000  | 00000000     | 00000000 |
| x30   t5  | x31   t6  |           |              |          |
| 00000000  | 00000000  |           |              |          |

**Floating point registers** [Hex]

| f0   ft0         | f1   ft1         | f2   ft2         | f3   ft3         | f4   ft4         |
|------------------|------------------|------------------|------------------|------------------|
| 0000000000000000 | 0000000000000000 | 0000000000000000 | 0000000000000000 | 0000000000000000 |
| f5   ft5         | f6   ft6         | f7   ft7         | f8   fs0         | f9   fs1         |
| 0000000000000000 | 0000000000000000 | 0000000000000000 | 0000000000000000 | 0000000000000000 |
| f10   fa0        | f11   fa1        | f12   fa2        | f13   fa3        | f14   fa4        |
| 0000000000000000 | 0000000000000000 | 0000000000000000 | 0000000000000000 | 0000000000000000 |
| f15   fa5        | f16   fa6        | f17   fa7        | f18   fs2        | f19   fs3        |
| 0000000000000000 | 0000000000000000 | 0000000000000000 | 0000000000000000 | 0000000000000000 |
| f20   fs4        | f21   fs5        | f22   fs6        | f23   fs7        | f24   fs8        |
|                  |                  |                  |                  |                  |

## 2.1 CREATOR

- Simulador *genérico*
  - Definición de arquitecturas mediante **archivos de configuración**

```
- name: beq
  template: B
  fields:
    - field: opcode
      value: "1100011"
    - field: funct3
      value: "000"
  definition: |
    if (registers[rs1] == registers[rs2])
      registers.pc = registers.pc + imm;
  help: Take the branch if registers rs1 and rs2 are equal.
```

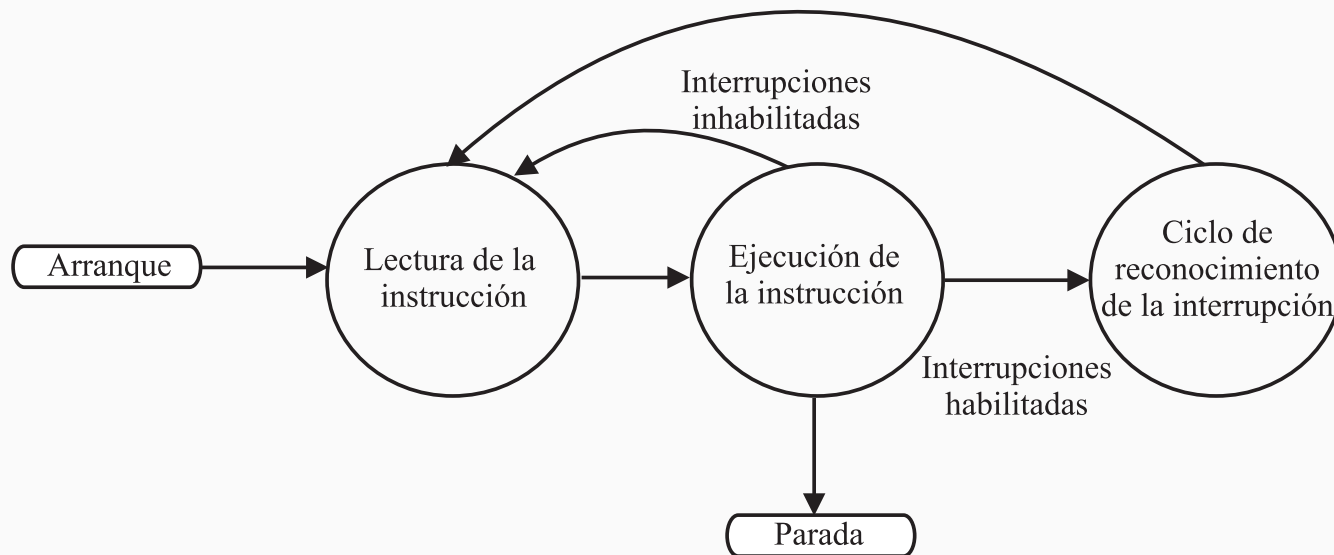
YAML

- Aplicación web y CLI

## 2.2 Interrupciones

### Evento no programado que altera la ejecución del programa

- Interno o externo
- Diferentes tipos:
  - *Maskable/Non-maskable*
  - *Software/externos/excepciones*
  - Síncronos/asíncronos
- Pin/registro en la CPU para marcar interrupción



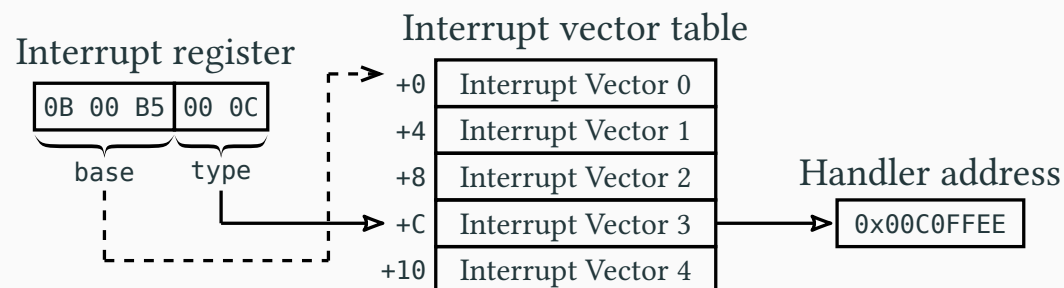
## 2.2 Interrupciones

### Gestión de interrupciones:

1. Guardar contexto
2. Cambio modo ejecución
3. Tratar interrupción
4. Restaurar contexto

### Modos de tratar interrupciones:

- *Polled*: *Handler* único
- *Vectored*: Tabla de interrupciones





### Implementación de interrupciones

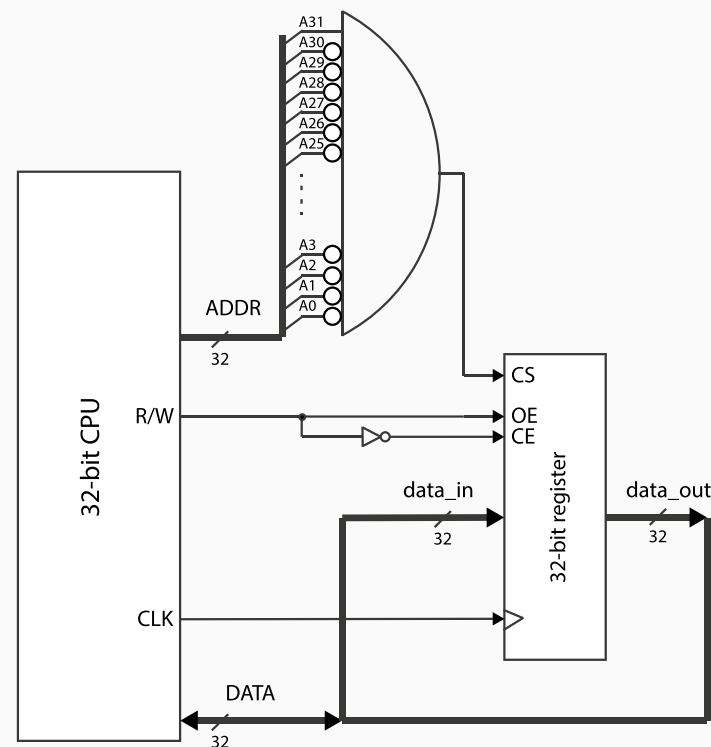
- Dependiente de ISA
- Simuladores:
  - ISAs simples (e.g. Z80): implementan correctamente
  - ISAs complejas (e.g. RISC-V): sólo en «profesionales»
  - Simuladores genéricos:
    - CREATOR 5: Soporte inicial (*polled*) en MIPS
    - WepSIM: Programable, *flag* específica

## 2.3 Temporizadores

- Fundamental para S.O., comunicación, etc.
- Temporizadores *software* ineficientes
- Contador que avanza cada *tick* → múltiplo de oscilador
- Genera interrupción cuando llega a valor programado
- Suelen implementarse en simuladores

## 2.4 Dispositivos de E/S

- Comunicación a través de memoria
  - Típicamente registros *on-device*
- Dos formas principales de direccionamiento:
  - «Puertos»: Requiere instrucciones específicas (*in/out*)
  - *Memory-mapped*: Se usa porción de memoria del sistema



### 3. Diseño

---

- Nuevos módulos en ejecutor:
  1. Interrupt Manager
  2. Timer Manager
  3. Device Manager
- Modificar ciclo ejecución de instrucción

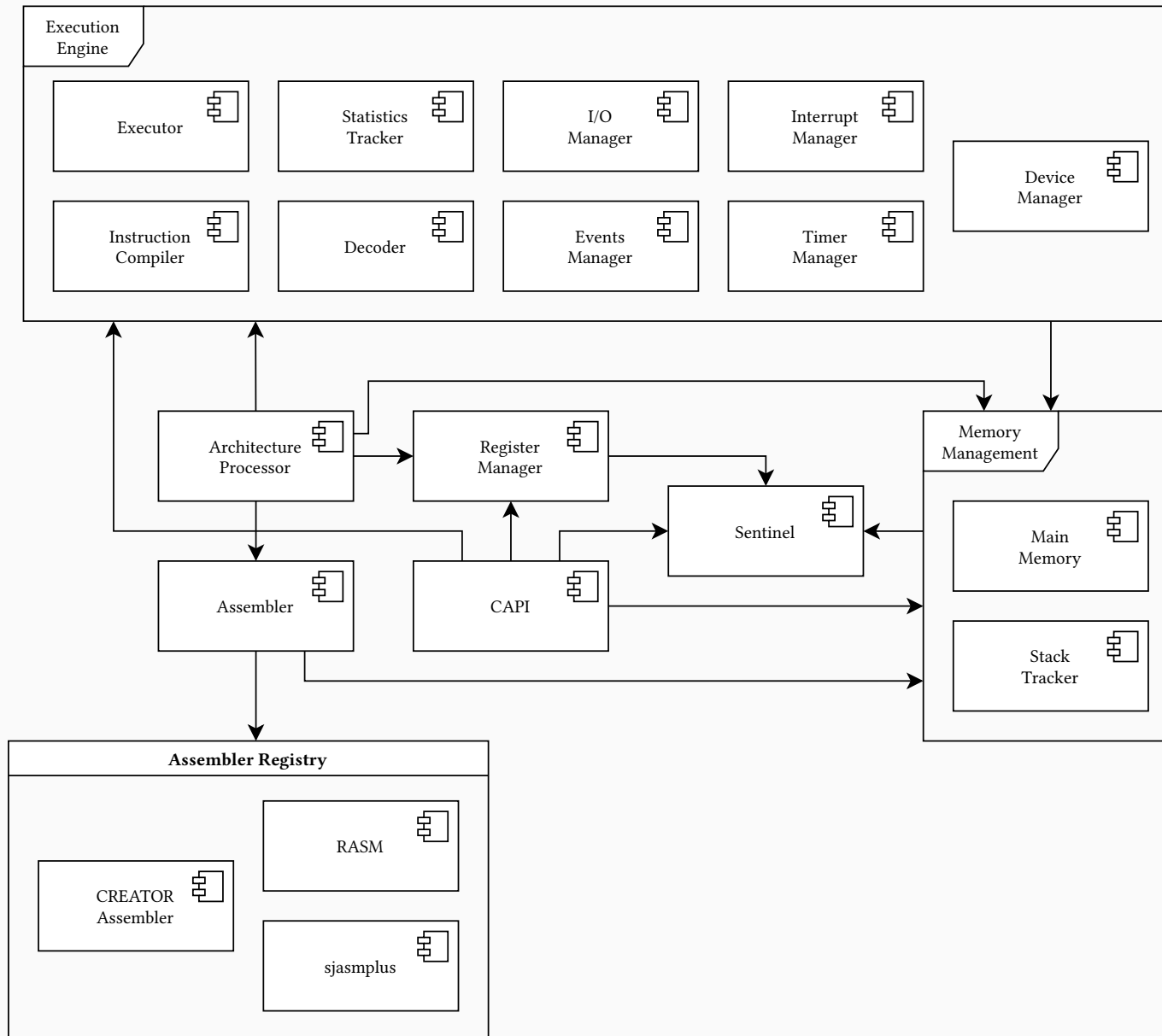
---

### INSTRUCTION EXECUTION CYCLE

---

- 1: Decode *instruction*
  - 2: Increment PC
  - 3: Execute *instruction*
  - 4: Handle timers
  - 5: Handle devices
  - 6: Handle interrupts
  - 7: Fetch *instruction*
-

## 3.1 Arquitectura



## 3.2 Interrupciones

Modelo «genérico» de interrupciones.

- Conjunto de acciones comunes
  - check, create
  - enable, disable, global\_enable, global\_disable
  - is\_enabled, is\_disabled, is\_global\_enabled, is\_global\_disabled
  - clear, global\_clear
- Definidos en configuración de arquitectura
- Conjunto de tipos de interrupciones
  - Software, Timer, External, EnvironmentCall, Maskable, NonMaskable

```
interrupts:
```

YAML

```
  check: |
```

```
    if (registers.mip & (2n ** 11n)) return InterruptType.External;
```

```
    if (registers.mip & (2n ** 3n)) return InterruptType.Software;
```

```
    if (registers.mip & (2n ** 7n)) return InterruptType.Timer;
```

```
    return null;
```

```
# ...
```

## 3.2 Interrupciones

---

### INTERRUPT HANDLING SUBROUTINE

---

```
1: if not is_global_enabled() then
2:   | return
   end if
3: pending_interrupt ← check()
4: if not pending_interrupt then
5:   | return
   end if
6: if not is_enabled(pending_interrupt) then
7:   | return
   end if
8: handle(pending_interrupt)
```

---



## 3.2 Interrupciones

### Llamadas al sistema

Funciones proporcionadas por el S.O. → requieren privilegio

- Generar interrupción
- Escribir por pantalla, etc.

En CREATOR, necesario emular un S.O.

- No se quiere *obligar* a implementar

Dos soluciones:

1. Kernel en ensamblador «escondido» → usuario puede sobreescribirlo
2. Doble *handler* de interrupciones en JavaScript:
  1. *Handler* por defecto: ignora interrupciones, sólo atiende *syscalls* (definidas en arquitectura)
  2. *Handler* «custom»: ejecuta conforme a modelo de interrupciones → requiere implementar RTI
    - Usuario elige cual usar
    - Sincronizar *handlers*
    - Facilmente extensible
    - Mayor control para interactuar con el sistema

## 3.2 Interrupciones

custom: |

YAML

```
CAPI.INTERRUPTS.globalDisable();
CAPI.INTERRUPTS.setKernelMode();
registers.epc = registers.pc;

// jump to handler
if (registers.mtvec & 1n) { // vectored mode
    registers.pc =
        (registers.mtvec >> 2n) + 4 * (registers.mcause & (2 ** 32 -
            1));
} else { // direct mode
    registers.pc = registers.mtvec >> 2n
}
```

## 3.2 Interrupciones

```
creator_syscall: |
```

YAML

```
  let [fa0Value, fa0Type] = CAPI.ARCH.toJSNumberD(registers.fa0);
```

```
  switch (registers.a7) {
```

```
    case 1n:
```

```
      CAPI.SYSCALL.print(registers.a0, 'int32');
```

```
      break;
```

```
    case 2n:
```

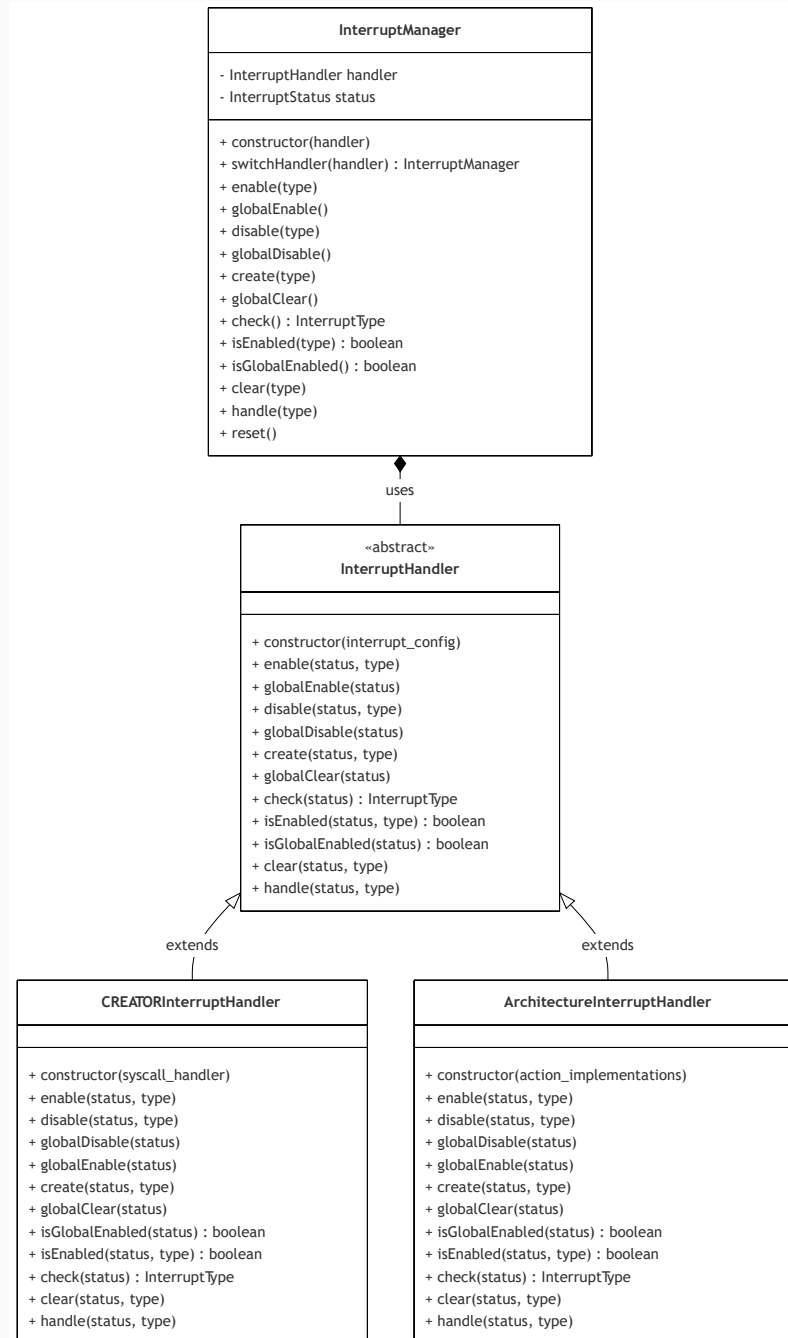
```
    case 3n:
```

```
      CAPI.SYSCALL.print(fa0Value, 'float');
```

```
      break;
```

```
    // ...
```

## 3.2 Interrupciones



## 3.3 Temporizadores

Modelo «genérico»:

- `tick_cycles`
- `advance`
- `handler`
- `is_enabled`, `enable`, `disable`

```
timer:
  tick_cycles: 1
  advance: |
    registers.mtime = (registers.mtime + 1n) % (2n**32n - 1n);
  handler: |
    if (registers.mtime === registers.mtimecmp) {
      CAPI.INTERRUPTS.create(InterruptType.Timer);
    }
  # ...
```

YAML

## 3.3 Temporizadores

---

### TIMER HANDLING SUBROUTINE

---

```
1: if not is_enabled() then
2:   | return
   end if
3: if clk_cycles % tick_cycles = 0 then
4:   | advance()
5:   | handle()
   end if
```

---

## 3.4 Dispositivos direccionados en memoria

Modelo «genérico» de dispositivo:

- Registro de control
- Registro de estado
- Segmento de datos
- *Handler*

Implementados en la aplicación

- Consola (lectura/escritura), e interacción con el sistema (apagado, etc.)
- Direcciones de memoria definidas en arquitectura

---

### DEVICE HANDLING SUBROUTINE

---

```
1: for-each device in devices
2:   if not device.enabled() then
3:     | continue
4:   end if
   device.handler()
end for-each
```

---

## 4. Implementación

---





**ESLint**

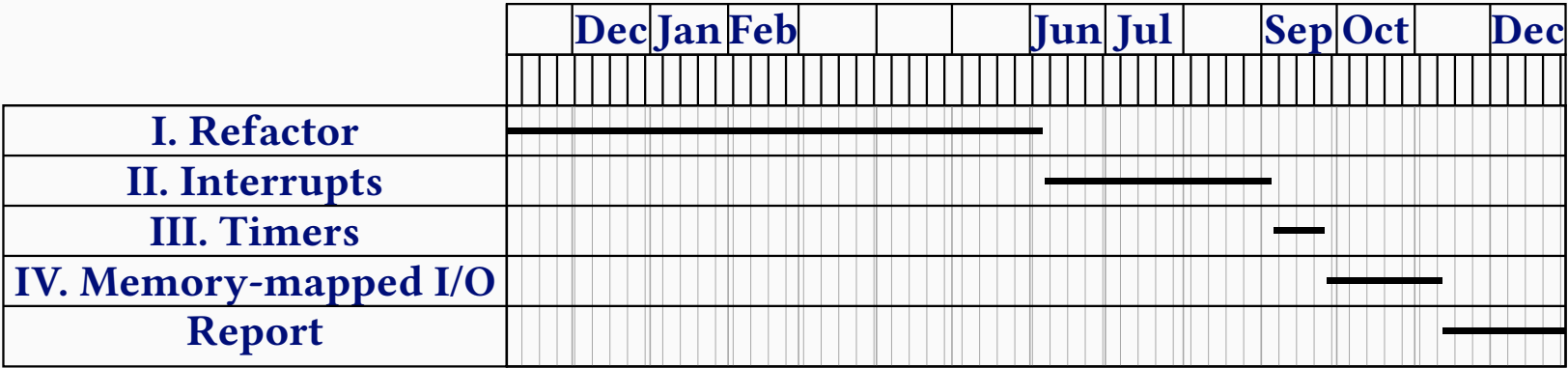


**Prettier**



**Vite**

# 4.2 Planificación



| Concepto              | Coste              |
|-----------------------|--------------------|
| Recursos humanos      | 20.600,00 €        |
| Recursos materiales   | 162,01 €           |
| Costes indirectos     | 298,64 €           |
| <b>Costes totales</b> | <b>21.060,65 €</b> |
| Incrementos           | 14.106,42 €        |
| <b>Importe final</b>  | <b>35.167,07 €</b> |

## 5. Conclusiones

---

## 5.1 Conclusiones del proyecto

Objetivos cumplidos:

- Implementaciones genéricas de interrupciones, temporizadores, y dispositivos E/S
- Llamadas al sistema «realistas»
- Código modular y fácilmente extensible

Conocimientos adquiridos:

- JavaScript, TypeScript, Vue.js
- Gestión de proyectos
- Refactorizado

## 5.2 Trabajos futuros

- Múltiples interrupciones por ciclo
- Interfaz gráfica para dispositivos
- Nuevos dispositivos (LEDs, etc.)
- Soporte para kernel
- Múltiples temporizadores (de forma más explícita)

¡Muchas gracias por su atención!